

## Workcells

A common use case is robots performing deliveries within facilities, so a `Delivery` task is configured into the `rmf_fleet_adapters`. In a delivery task, a payload is loaded onto the robot at one location (pickup waypoint) and unloaded at another (dropoff waypoint). The loading and unloading of the payload onto and from a robot may be automated by robots/workcells in the facility. These devices are henceforth referred to as dispensers and ingestors respectively.

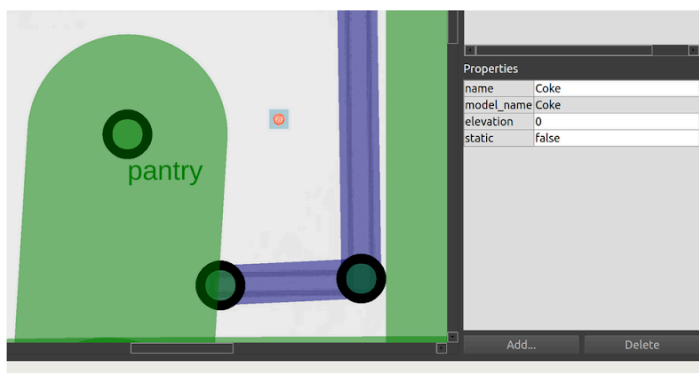
To replicate the loading and unloading processes in simulation, the `TeleportDispenser` and `TeleportIngestor` plugins have been designed. These plugins are attached to the `TeleportDispenser` and `TeleportIngestor` 3D models, respectively. To setup a payload loading station in simulation:

- Add a `TeleportDispenser` model beside the pickup waypoint and assign it a unique `name`
- Add the payload model beside the `TeleportDispenser` model (Coke can in image below)

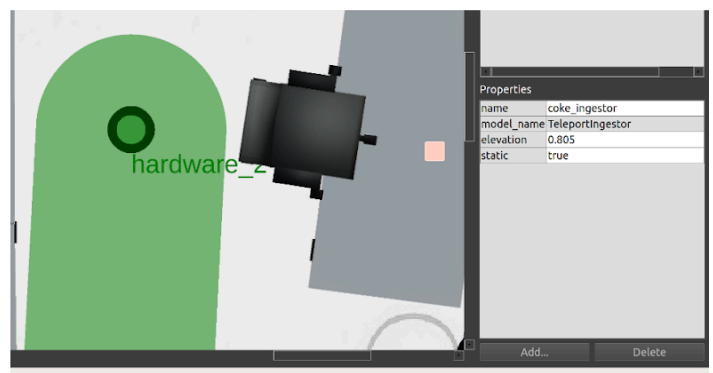
To setup a payload unloading station in simulation:

- Add a `TeleportIngestor` model beside the dropoff waypoint and assign it a unique `name`

When a `DispenserRequest` message is published with `target_guid` matching the name of the `TeleportDispenser` model, the plugin will teleport the payload onto the nearest robot model. Conversely, when an `IngestorRequest` message is published with the `target_guid` matching the name of the `TeleportIngestor` model, the `TeleportIngestor` plugin will teleport the payload from the robot to its location in the world. The combinations of these plugins allow delivery requests to be simulated. In the future, these mechanisms will be replaced by actual workcells or robot arms but the underlying message exchanges will remain the same.



TeleportDispenser model (highlighted in pink) with its payload nearby (beverage can). The model is placed beside the delivery pickup waypoint



TeleportIngestor model (highlighted in pink) placed beside the dropoff waypoint

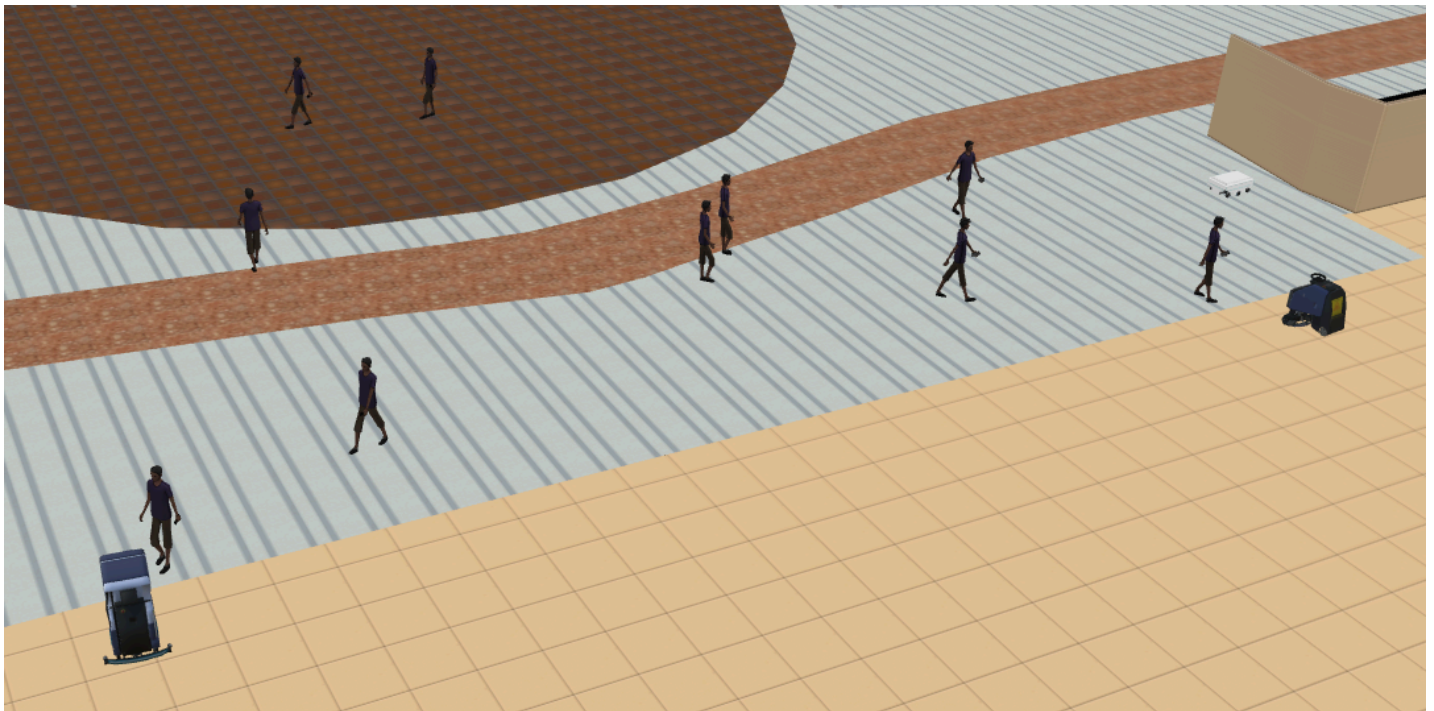
## Crowdsim

Crowd Simulation, aka `CrowdSim` is an optional feature in RMF simulation. User can choose to enable crowdsim on `rmf_traffic_editor`. In RMF, the crowdsim plugin uses [menge](#) as the core to control each of simulated agent in the world.

An example of crowdsim is demonstrated on `rmf_demos`'s `airport_world`:

```
ros2 launch rmf_demos_gz airport_terminal.launch.xml use_crowdsim:=1
```

For more details on how `crowdsim` works and how to configure it, please dive in to [the detailed guide for using Crowdsim](#).



## Creating Simulations and Running Scenarios

The section aims to provide an overview of the various components in the `rmf_demos` [repository](#) which may serve as a reference for setting up other simulations and assigning tasks to robots. Here, we will focus on the `office` world.

## Map package

The `rmf_demos_maps` package houses annotated `traffic_editor` files which will be used for the 3D world generation. Opening the `office.project.yaml` file in `traffic_editor` reveals a single level floorplan that has walls, floors, scale measurements, doors, lanes and models annotated. All the robot lanes are set to `bidirectional` with `graph_idx` equal to "0". The latter signifies that all the lanes belong to the same fleet. In the `airport` world, we have two sets of graphs with indices "0" and "1" which reflect laneways occupiable by two fleets respectively. The figure below highlights properties assigned to a lane and a waypoint that serves as a robot spawn location.



Robot spawn location properties

To export a 3D world file along with the navigation graphs, the `building_map_generator` script is used. The `CMakeLists.txt` file of this package is configured to automatically run the generator scripts when the package is built. The outputs are installed to the `share/` directory for the package. This allows for the generated files to be easily located and used by other packages in the demo.

```

foreach(path ${traffic_editor_paths})

  # Get the output world name
  string(REPLACE "." ";" list1 ${path})
  list(GET list1 0 name)
  string(REPLACE "/" ";" list2 ${name})
  list(GET list2 -1 world_name)

  set(map_path ${path})
  set(output_world_name ${world_name})
  set(output_dir ${CMAKE_CURRENT_BINARY_DIR}/maps/${output_world_name})
  set(output_world_path ${output_dir}/${output_world_name}.world)
  set(output_model_dir ${output_dir}/models)

  # first, generate the world
  add_custom_command(
    OUTPUT ${output_world_path}
    COMMAND ros2 run rmf_building_map_tools building_map_generator gazebo
    ${map_path} ${output_world_path} ${output_model_dir}
    DEPENDS ${map_path}
  )

  add_custom_target(generate_${output_world_name} ALL
    DEPENDS ${output_world_path}
  )

  # now, generate the nav graphs
  set(output_nav_graphs_dir ${output_dir}/nav_graphs/)
  set(output_nav_graphs_phony ${output_nav_graphs_dir}/phony)
  add_custom_command(
    OUTPUT ${output_nav_graphs_phony}
    COMMAND ros2 run rmf_building_map_tools building_map_generator nav ${map_path}
    ${output_nav_graphs_dir}
    DEPENDS ${map_path}
  )

  add_custom_target(generate_${output_world_name}_nav_graphs ALL
    DEPENDS ${output_nav_graphs_phony}
  )

  install(
    DIRECTORY ${output_dir}
    DESTINATION share/${PROJECT_NAME}/maps
  )

endforeach()

```

## Launch Files

The `rmf_demos` package includes all the essential launch files required to bring up the simulation world and start various RMF services. The office simulation is launched using the `office.launch.xml` file. First, a `common.launch.xml` file is loaded and starts:

- The `rmf_traffic_schedule` node responsible for maintaining the database of robot trajectories and monitoring traffic for conflicts. If a conflict is detected, notifications are sent to relevant fleet adapters which begin the negotiation process to find an optimal resolution.
- The `building_map_server` which publishes a `BuildingMap` message used by UIs for visualization. The executable takes in the path to the relevant `.building.yaml` file as an argument. The `office.building.yaml` file installed by the `rmf_demos_maps` package is located using the `find-pkg-share` substitution command and is stored in the `config_file` argument.
- The `rmf_schedule_visualizer` which is an RViz based UI to visualize the traffic lanes, actual positions of the robots, expected trajectory of robots as reflected in the `rmf_traffic_schedule` and states of building systems such as door and lifts.
- The `door_supervisor` and `lift_supervisor` nodes to manage requests submitted by fleet adapter and UIs.

```
<!-- Common launch -->
<include file="$(find-pkg-share demos)/common.launch.xml">
  <arg name="use_sim_time" value="true"/>
  <arg name="viz_config_file" value="$(find-pkg-share
demos)/include/office/office.rviz"/>
  <arg name="config_file" value="$(find-pkg-share
rmf_demos_maps)/office/office.building.yaml"/>
</include>
```

To launch a simulated world in gazebo, a snippet from [rmf\\_demos\\_gz](#) is shown below. Similarly, user can also choose to run with ignition simulator, [rmf\\_demos\\_ign](#)

```

<group>
  <let name="world_path" value="$(find-pkg-share
rmf_demos_maps)/maps/office/office.world" />
  <let name="model_path" value="$(find-pkg-share
rmf_demos_maps)/maps/office/models:$(find-pkg-share
rmf_demos_assets)/models:/usr/share/gazebo-9/models" />
  <let name="resource_path" value="$(find-pkg-share
rmf_demos_assets):/usr/share/gazebo-9" />
  <let name="plugin_path" value="$(find-pkg-prefix
rmf_gazebo_plugins)/lib:$(find-pkg-prefix building_gazebo_plugins)/lib" />

  <executable cmd="gzserver --verbose -s libgazebo_ros_factory.so -s
libgazebo_ros_init.so $(var world_path)" output="both">
    <env name="GAZEBO_MODEL_PATH" value="$(var model_path)" />
    <env name="GAZEBO_RESOURCE_PATH" value="$(var resource_path)" />
    <env name="GAZEBO_PLUGIN_PATH" value="$(var plugin_path)" />
    <env name="GAZEBO_MODEL_DATABASE_URI" value="" />
  </executable>
  <executable cmd="gzclient --verbose $(var world_path)" output="both">
    <env name="GAZEBO_MODEL_PATH" value="$(var model_path)" />
    <env name="GAZEBO_RESOURCE_PATH" value="$(var resource_path)" />
    <env name="GAZEBO_PLUGIN_PATH" value="$(var plugin_path)" />
  </executable>
</group>

```

Lastly, instances of the "full control" `rmf_fleet_adapter` are launched for each robot type annotated in the map. The navigation graphs for each fleet as generated by the `building_map_generator` script is passed via the `nav_graph_file` argument. For the office map, a single fleet of Magni robots is defined. Hence, a single `magni_adapter.launch.xml` file configured with the kinematic properties of this robot type along with spatial thresholds used for planning, is launched. Along with the fleet adapter, a `robot_state_aggregator` node is started. This node aggregates `RobotState` messages with `RobotState.name` containing the `robot_prefix` argument and publishes the aggregate to `/fleet_states` with `FleetState.name` specified by the `fleet_name` argument.

```

<group>
  <let name="fleet_name" value="magni"/>
  <include file="$(find-pkg-share
rmf_demos)/include/adapters/magni_adapter.launch.xml">
    <arg name="fleet_name" value="$(var fleet_name)"/>
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="nav_graph_file" value="$(find-pkg-share
rmf_demos_maps)/maps/office/nav_graphs/0.yaml" />
  </include>
  <include file="$(find-pkg-share
rmf_fleet_adapter)/robot_state_aggregator.launch.xml">
    <arg name="robot_prefix" value="magni"/>
    <arg name="fleet_name" value="$(var fleet_name)"/>
    <arg name="use_sim_time" value="true"/>
  </include>
</group>

```

When testing RMF with hardware, the same launch files can be used, with the exception of starting `Gazebo`. More information on running demos with hardware can be found [the chapter on Integration](#).

## Task Requests

RMF supports various tasks out of the box. For more information see [Tasks in RMF](#). A web-based dashboard is provided to allow users to send commands to RMF. Once the [dashboard server](#) is launched, it can be accessed at <https://open-rmf.github.io/rmf-panel-js/>.

The screenshot displays the RMF Panel web interface. At the top, there's a navigation bar with 'Office', 'Airport', and 'Clinic' tabs. The main header shows 'Time is: 0:1:37' and 'Task Submissions'. Below this, there are two sections: 'Submit a Task' and 'Submit a List of Tasks'. The 'Submit a Task' section has a dropdown for 'Select a request type', a text input for 'Set start time (mins from now)' with the value '0', and a dropdown for 'Choose an evaluator (optional)'. The 'Submit a List of Tasks' section has a 'Choose file' button, a text area containing a JSON task list, and a 'SUBMIT TASK LIST' button. Below these sections is the 'Robots & Tasks Summaries' section. It is divided into two parts: 'Robots' and 'Tasks'. The 'Robots' part shows a 'REFRESH' button and two robot cards: 'tinyRobot1' (Task ID: Loop0, Status: Moving-2, Battery: 99.88%, Location: L1) and 'tinyRobot2' (Task ID: Delivery1, Status: Moving-2, Battery: 99.89%, Location: L1). The 'Tasks' part shows a 'REFRESH' button and three task cards. Each task card displays a progress gauge, task details, robot information, task type, task state, and start/end times. Task 1 (Loop0) is 7% complete, Task 2 (Delivery1) is 54% complete, and Task 3 (Loop2) is 0% complete. Each task card has a 'CANCEL TASK' button.

**RMF Panel**

Office Airport Clinic

Time is: 0:1:37

### Task Submissions

**Submit a Task**

Select a request type

Set start time (mins from now)

0

Choose an evaluator (optional)

**Submit a List of Tasks**

Choose file office\_tasks.json

```

eg. [
  {"task_type": "Loop", "start_time": 0, "description": {"num_loops": 5, "start_name": "coe", "finish_name": "lounge"}},
  {"task_type": "Delivery", "start_time": 0, "description": {"option": "coke"}},
  {"task_type": "Loop", "start_time": 0, "description": {"num_loops": 5, "start_name": "cubicle_2", "finish_name": "supplies"}}
]

```

SUBMIT TASK LIST

### Robots & Tasks Summaries

**Robots**

REFRESH

| Robot      | Task ID   | Status   | Battery | Location |
|------------|-----------|----------|---------|----------|
| tinyRobot1 | Loop0     | Moving-2 | 99.88%  | L1       |
| tinyRobot2 | Delivery1 | Moving-2 | 99.89%  | L1       |

**Tasks**

REFRESH

| Task ID   | Details                  | Robot      | Task Type | Task State       | Start | End |
|-----------|--------------------------|------------|-----------|------------------|-------|-----|
| Loop0     | coe -> lounge            | tinyRobot1 | Loop      | Active/Executing | 55    | 585 |
| Delivery1 | pantry -> hardware_2     | tinyRobot2 | Delivery  | Active/Executing | 58    | 130 |
| Loop2     | cubicle_2 -> supplies x5 | tinyRobot2 | Loop      | Queued           | 692   | 0   |

Developed by Open Robotics 2020  
RMF Demo Panel is Powered by RMF

Alternatively several scripts exist in `rmf_demos_tasks` to assist users with submitting requests from the terminal. Presently the `dispatch_loop.py`, `dispatch_delivery.py` and `dispatch_clean.py` scripts can be used to submit `Loop`, `Delivery` and `Clean` requests.

## Conclusion

This chapter covered the utilization of the `traffic_editor` tool to create annotated maps that allow the auto-generation of 3D worlds for simulations. It also covered the assets used within simulations and the corresponding plugins necessary for ROS 2 and RMF to interface with them. A working example of these components running together, in the form of the `rmf_demos_maps` package, was provided as a reference for how to actualize a custom system. The next chapter will introduces the basic concept behind RMF.

---

Found a bug? [Edit this page on GitHub](#).

# RMF Core Overview

This chapter describes RMF, an umbrella term for a wide range of open specifications and software tools that aim to ease the integration and interoperability of robotic systems, building infrastructure, and user interfaces. `rmf_core` consists of:

- `rmf_traffic`: Core scheduling and traffic management systems
- `rmf_traffic_ros2`: `rmf_traffic` for `ros2`
- `rmf_task`: Task planner for `rmf`
- `rmf_battery`: `rmf` battery estimation
- `rmf_ros2`: `ros2` adapters and nodes and python bindings for `rmf_core`
- `rmf_utils`: utility for `rmf`

## Traffic deconfliction

Avoiding mobile robot traffic conflicts is a key functionality of `rmf_core`. There are two levels to traffic deconfliction: (1) prevention, and (2) resolution.

### Prevention

Preventing traffic conflicts whenever possible is the best-case scenario. To facilitate traffic conflict prevention, we have implemented a platform-agnostic Traffic Schedule Database. The traffic schedule is a living database whose contents will change over time to reflect delays, cancellations, or route changes. All fleet managers that are integrated into an RMF deployment must report the expected itineraries of their vehicles to the traffic schedule. With the information available on the schedule, compliant fleet managers can plan routes for their vehicles that avoid conflicts with any other vehicles, no matter which fleet they belong to. `rmf_traffic` provides a `Planner` class to help facilitate this for vehicles that behave like standard AGVs (Automated Guided Vehicles), rigidly following routes along a pre-determined grid. In the future we intend to provide a similar utility for AMRs (Autonomous Mobile Robots) that can perform ad hoc motion planning around unanticipated obstacles.

### Negotiation

It is not always possible to perfectly prevent traffic conflicts. Mobile robots may experience delays because of unanticipated obstacles in their environment, or the predicted schedule may be flawed for any number of reasons. In cases where a conflict does arise, `rmf_traffic` has a

Negotiation scheme. When the Traffic Schedule Database detects an upcoming conflict between two or more schedule participants, it will send a conflict notice out to the relevant fleet managers, and a negotiation between the fleet managers will begin. Each fleet manager will submit its preferred itineraries, and each will respond with itineraries that can accommodate the others. A third-party judge (deployed by the system integrator) will choose the set of proposals that is considered preferable and notify the fleet managers about which itineraries they should follow.

There may be situations where a sudden, urgent task needs to take place (for example, a response to an emergency), and the current traffic schedule does not accommodate it in a timely manner. In such a situation, a traffic participant may intentionally post a traffic conflict onto the schedule and force a negotiation to take place. The negotiation can be forced to choose an itinerary arrangement that favors the emergency task by implementing the third-party judge to always favor the high-priority participant.

## Traffic Schedule

The traffic schedule is a centralized database of all the intended robot traffic trajectories in a facility. Note that it contains the intended trajectories; it is looking into the future. The job of the schedule is to identify conflicts in the intentions of the different robot fleets and notify the fleets when a conflict is identified. Upon receiving the notification, the fleets will begin a traffic negotiation, as described above.